

Patrones de comportamiento

Caso: negocio de alquiler “Los atuendos”

Patrones de diseño de software

Estudiantes: Jose Manuel Castro López

Sergio Alejandro Parra Arango

Grupo: 2310-8B MOM 1 VIRTUAL

Ingeniería de Software

Marzo de 2025

Contenido

Patrones de diseño utilizados	4
Tabla 1. <i>Tabla de patrones de diseño y su contribución.</i>	4
Figura 1. <i>Diagrama UML reconstruido.</i>	6
Figura 2. <i>Diagrama de secuencia del patrón Factory Method.</i>	7
Figura 3. <i>Diagrama de secuencia del patrón Singleton.</i>	7
Figura 4. <i>Diagrama de secuencia del patrón Composite.</i>	8
Figura 5. <i>Diagrama de secuencia del patrón Adapter.</i>	8
Patrones de comportamiento	9
Tabla 2 <i>Detalle de Patrones de Comportamiento</i>	9
Figura 6. <i>Diagrama de secuencia del patrón de comportamiento Command (Crear alquiler).</i> 11	
Figura 7. <i>Diagrama de secuencia del patrón de comportamiento Obeserver (cambio de estado de prenda).</i>	12
Figura 8. <i>Diagrama de secuencia del patrón de comportamiento Strategy (Métodos de pago).</i>	13
Figura 9. <i>Diagrama de secuencia del patrón de comportamiento Iterator (recorrer prendas)</i> 14	
Conclusion	14

Resumen

El desarrollo de software moderno exige la construcción de sistemas que no solo cumplan con los requerimientos funcionales, sino que también sean flexibles, mantenibles y escalables a lo largo del tiempo. En este contexto, el uso de patrones de diseño se convierte en una práctica fundamental, ya que permite resolver problemas comunes de manera estructurada y reutilizable.

El presente trabajo se centra en el análisis, rediseño y mejora del sistema de alquiler “Los Atuendos”, partiendo de una versión inicial con limitaciones en su arquitectura y alto acoplamiento entre sus componentes. A partir de esta base, se identificaron oportunidades de mejora mediante la aplicación de patrones de diseño creacionales, estructurales y de comportamiento.

En una primera fase, se incorporaron patrones como Factory Method, Singleton, Composite, Adapter y Decorator, con el objetivo de mejorar la organización del sistema y la gestión de sus componentes. Posteriormente, se complementó el diseño con patrones de comportamiento como Observer, Strategy, Command e Iterator, enfocados en optimizar la interacción entre objetos y la ejecución de procesos de negocio.

Como resultado, se obtuvo un rediseño del sistema soportado por diagramas de clases y de secuencia, los cuales permiten evidenciar una arquitectura más desacoplada, modular y preparada para futuros cambios, garantizando una mejor calidad del software.

Patrones de diseño utilizados

En la siguiente tabla se mostrarán los patrones de creación y estructurales propuestas para el mejoramiento de la aplicación de moda.

Tabla 1. *Tabla de patrones de diseño y su contribución.*

Patron a utilizar	Tipo(creación o estructural)	Contribución al diseño
Factory method	Creacional	El patrón factory method permite centralizar y desacoplar las diferentes prendas del sistema y delegarlas a una fábrica especializada, facilitando la extensión del sistema en caso de que en un futuro se desee agregar un nuevo tipo de prenda, ya que solo será necesario crear una nueva clase y su correspondiente fabrica sin modificar lo demás del código.

Singleton	Creacional	El patrón singleton limita a una única instancia por clase lo cual permite que todos los componentes del sistema accedan a una misma instancia centralizada. Un ejemplo sería la clase “negocioAlquiler”, lo cual asegura que exista un único punto de control en el sistema desde donde se gestionen listas de clientes, empleados, prendas y servicios.
Composite	Estructural	Permite tratar de manera uniforme objetos individuales y grupos de objetos. En el sistema de alquiler nos es útil para manejar las prendas dentro de un servicio de alquiler. Este patrón nos facilita la gestión de colecciones de objetos y simplifica las operaciones sobre ellas.
Adapter	Estructural	El adaptador convierte la información de las prendas del sistema a un formato compatible con el servicio de lavandería, sin necesidad de modificar las clases existentes. En el sistema de alquiler se puede usar para integrar el proceso de envío de prendas a la lavandería con un sistema externo o con un formato de datos diferente.

Decorator	Estructural	Permite agregar características adicionales a un objeto de manera dinámica. En el sistema de alquiler lo podemos aplicar para manejar características adicionales de las prendas, como piedras, accesorios o piezas adicionales. En lugar de crear varias combinaciones de clases con el patrón Decorator podemos agregar estas propiedades de forma flexible mediante clases decoradoras.
-----------	-------------	---

Figura 1. Diagrama UML reconstruido.

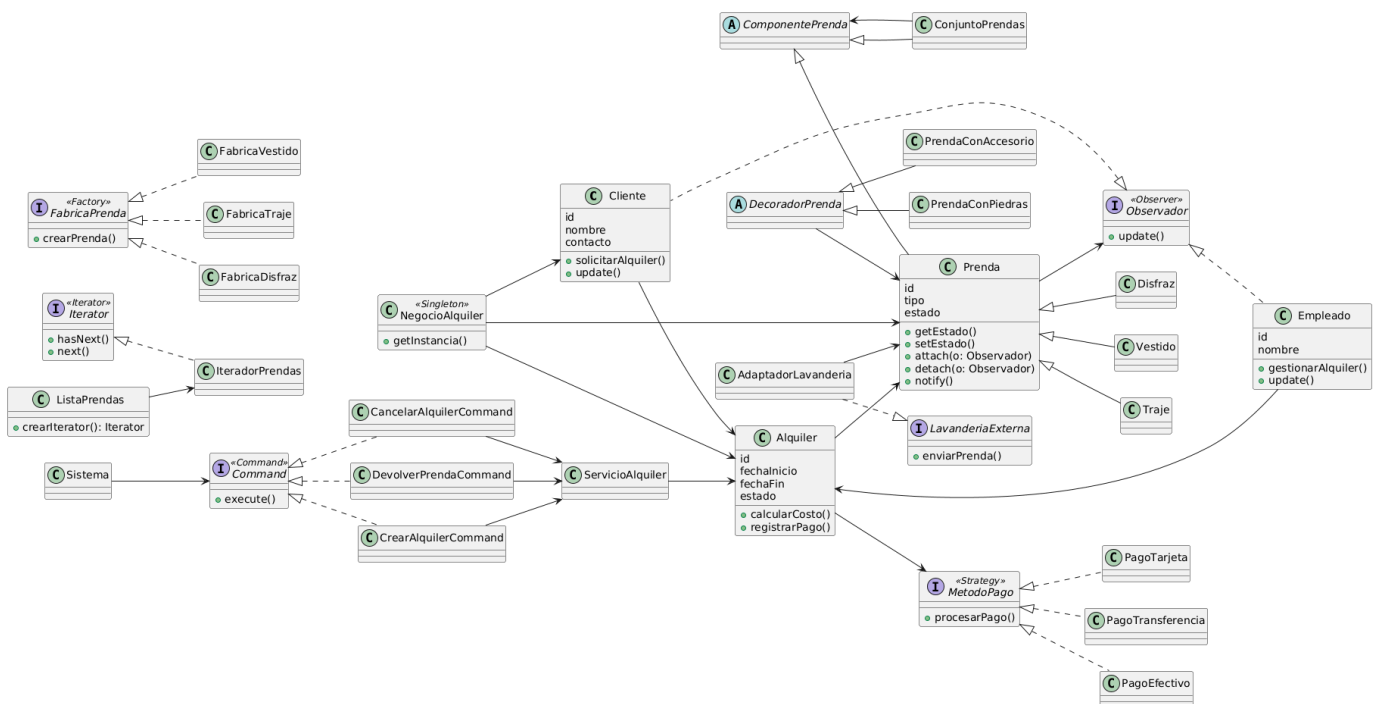


Figura 2. Diagrama de secuencia del patrón Factory Method.

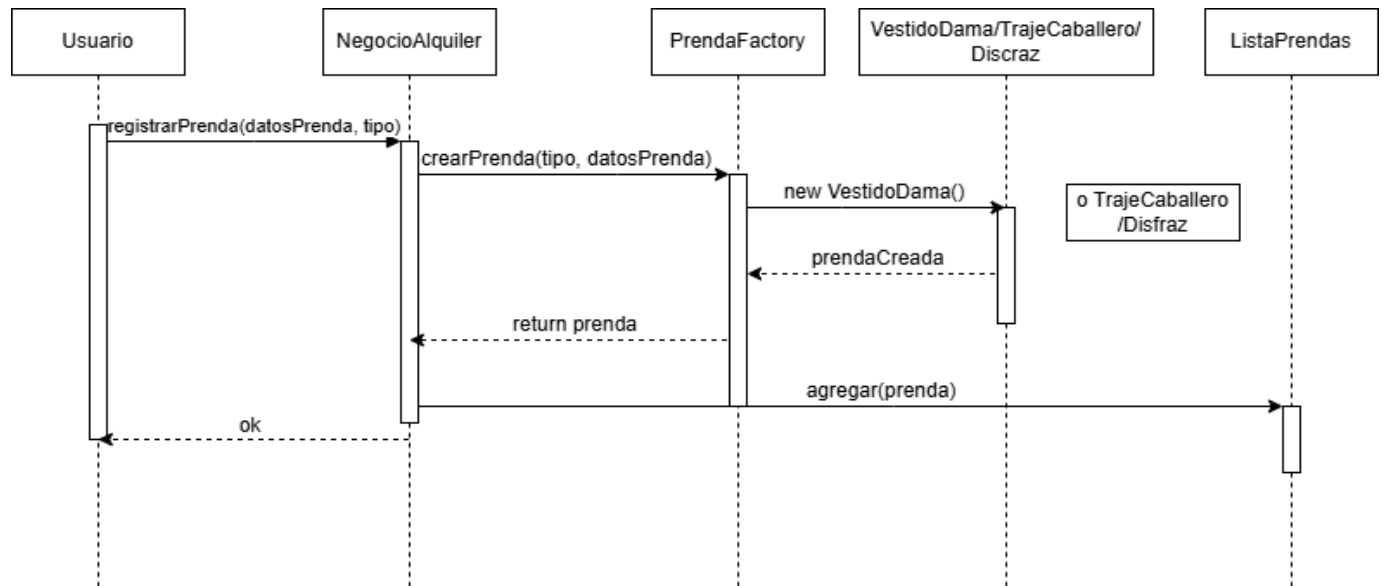


Figura 3. Diagrama de secuencia del patrón Singleton.

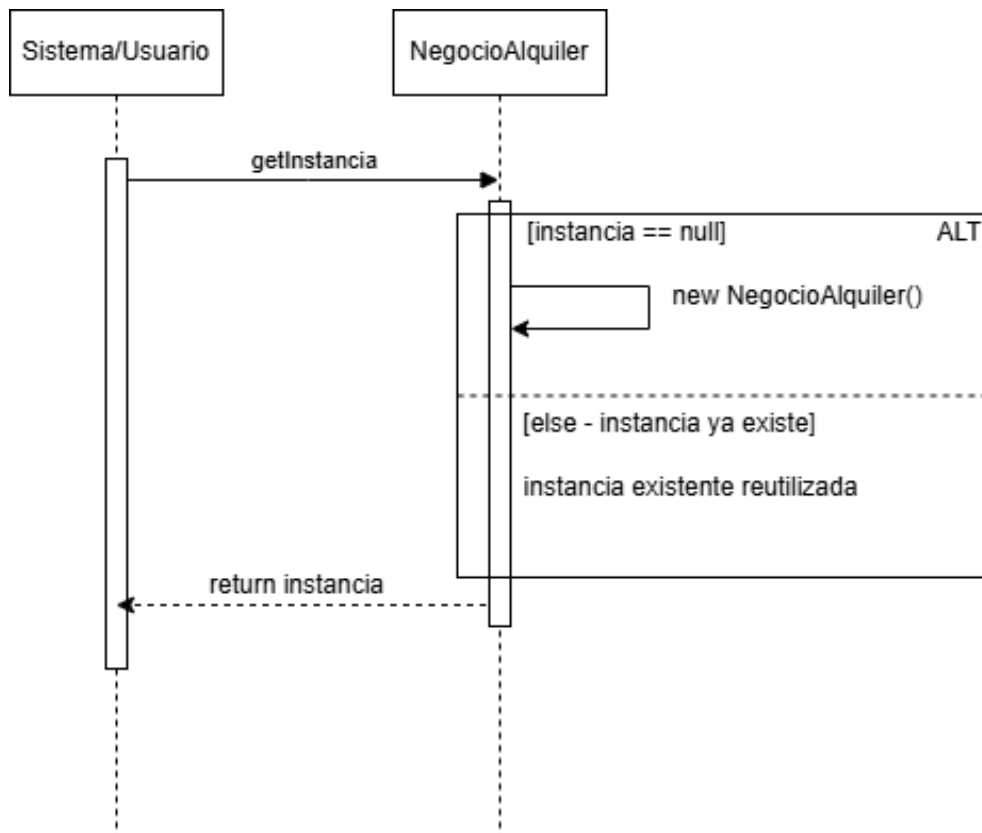


Figura 4. Diagrama de secuencia del patrón Composite.

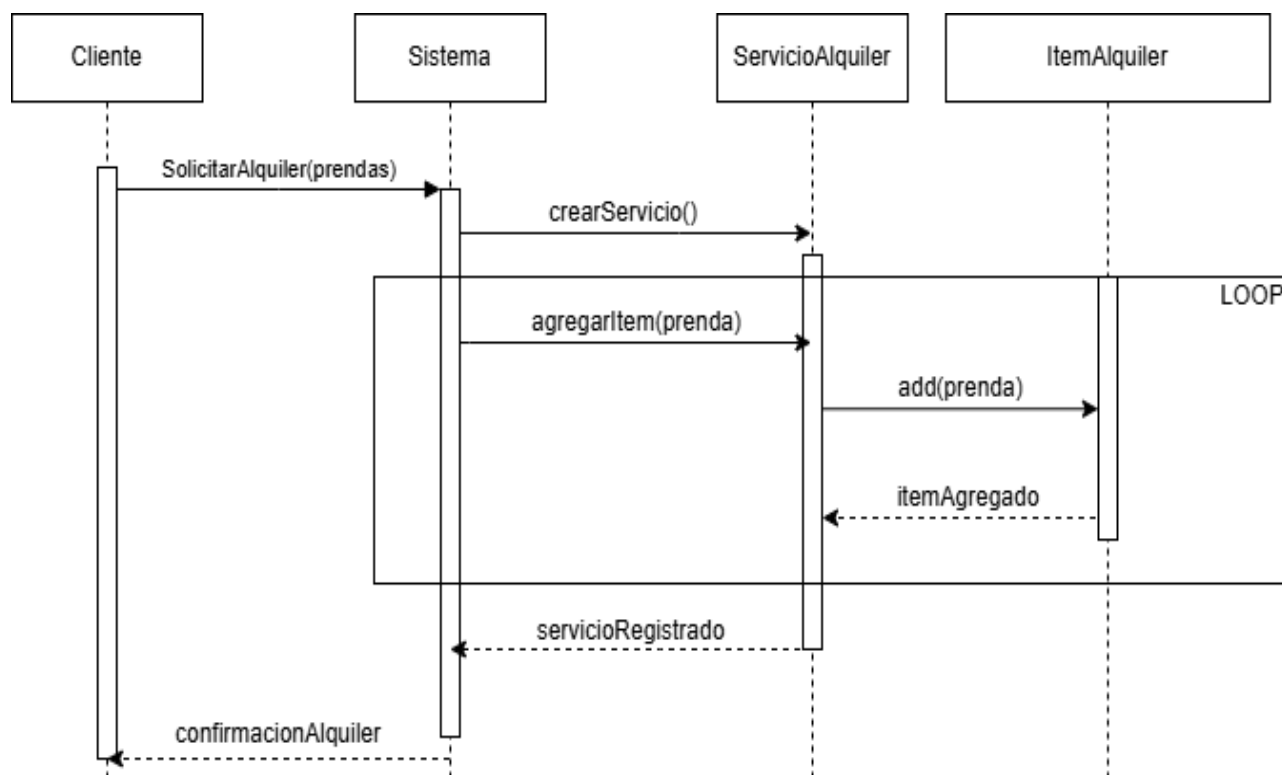
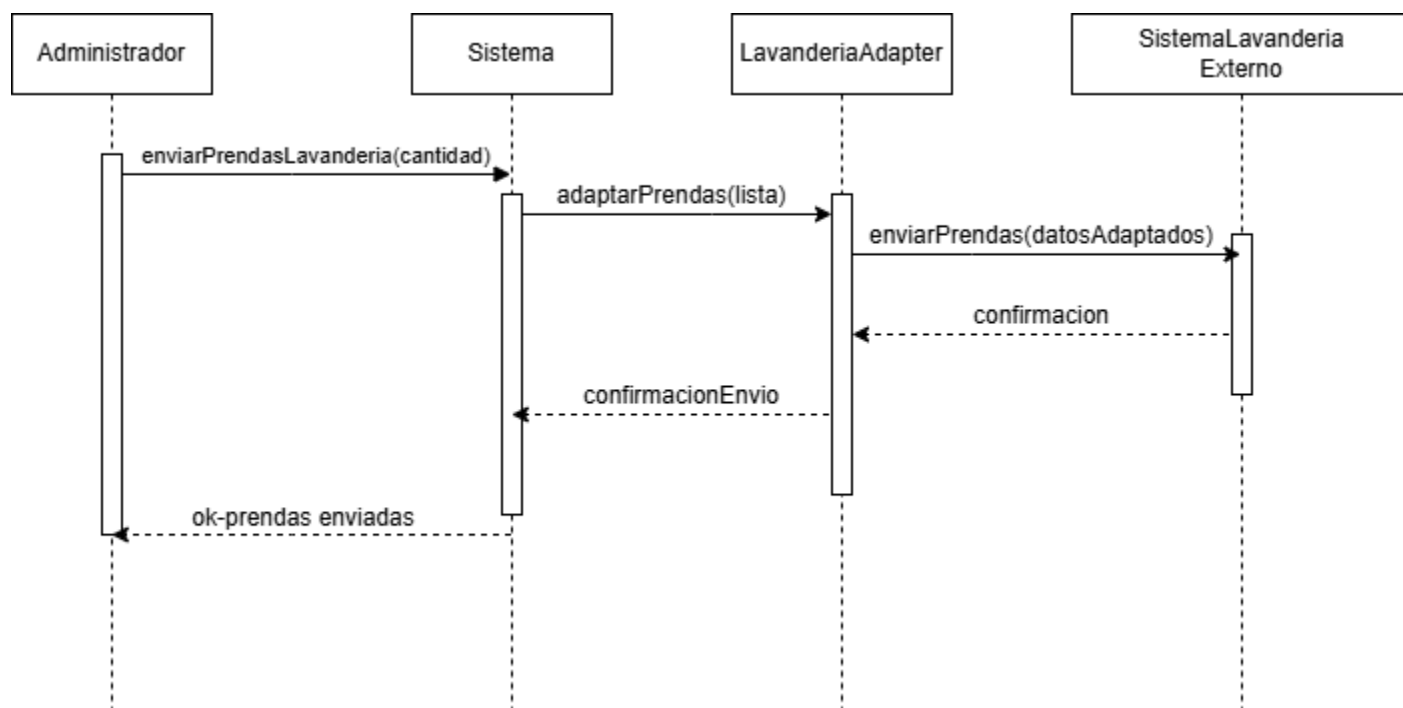


Figura 5. Diagrama de secuencia del patrón Adapter.



Patrones de comportamiento

Mientras que los patrones estructurales definen "cómo se organiza" el sistema, los Patrones de Comportamiento se eligen para gestionar "cómo interactúan" los objetos y cómo se reparte la responsabilidad de los procesos de negocio.

En el caso de "Los Atuendos", la elección de estos patrones busca:

- **Flexibilizar la Lógica de Negocio:** Permitir que reglas complejas, como la prioridad de lavado o el cálculo de fechas de alquiler, no estén "cableadas" en una sola clase, facilitando cambios futuros.
- **Optimizar la Comunicación entre Objetos:** Los diagramas de secuencia de comportamiento permiten visualizar el flujo de mensajes y asegurar que el sistema responda correctamente a eventos, como la devolución de una prenda o la consulta masiva de inventario.
- **Desacoplar Procesos:** Evitar que el sistema dependa de implementaciones rígidas para recorrer listas o ejecutar comandos, mejorando la mantenibilidad a largo plazo.

Tabla 2 *Detalle de Patrones de Comportamiento*

Patrón	Tipo	Aplicación en el sistema	Justificación
Observer	Comportamiento	Se utiliza para notificar a los clientes o empleados cuando cambia el estado de una	Permite desacoplar la lógica de notificación del sistema principal, facilitando la actualización

Patrón	Tipo	Aplicación en el sistema	Justificación
		prenda (disponible, alquilada, en lavandería).	automática de los usuarios interesados sin modificar otras clases.
Strategy	Comportamiento	Se aplica para gestionar diferentes métodos de pago en el sistema de alquiler (efectivo, tarjeta, transferencia).	Permite cambiar dinámicamente el algoritmo de pago sin modificar la lógica principal del sistema, mejorando la flexibilidad.
Command	Comportamiento	Se utiliza para encapsular acciones como crear alquiler, cancelar alquiler o devolver prenda.	Permite desacoplar el objeto que invoca la acción del que la ejecuta, facilitando la implementación de historial de acciones o deshacer operaciones.
Iterator	Comportamiento	Se aplica para recorrer colecciones como listas de prendas, clientes o alquileres.	Facilita el acceso secuencial a los elementos sin exponer la estructura interna de las colecciones.

Figura 6. Diagrama de secuencia del patrón de comportamiento Command (Crear alquiler).

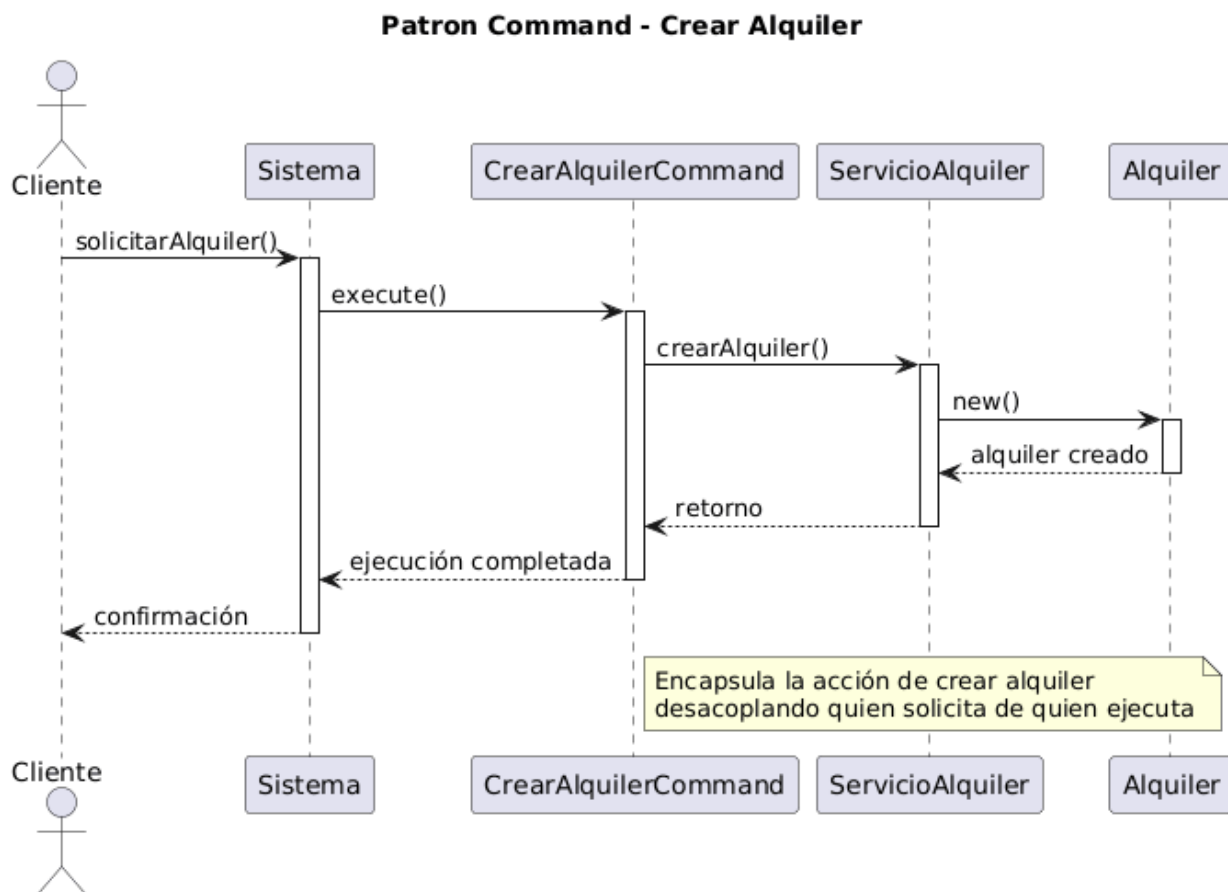


Figura 7. Diagrama de secuencia del patrón de comportamiento Observer (cambio de estado de prenda).

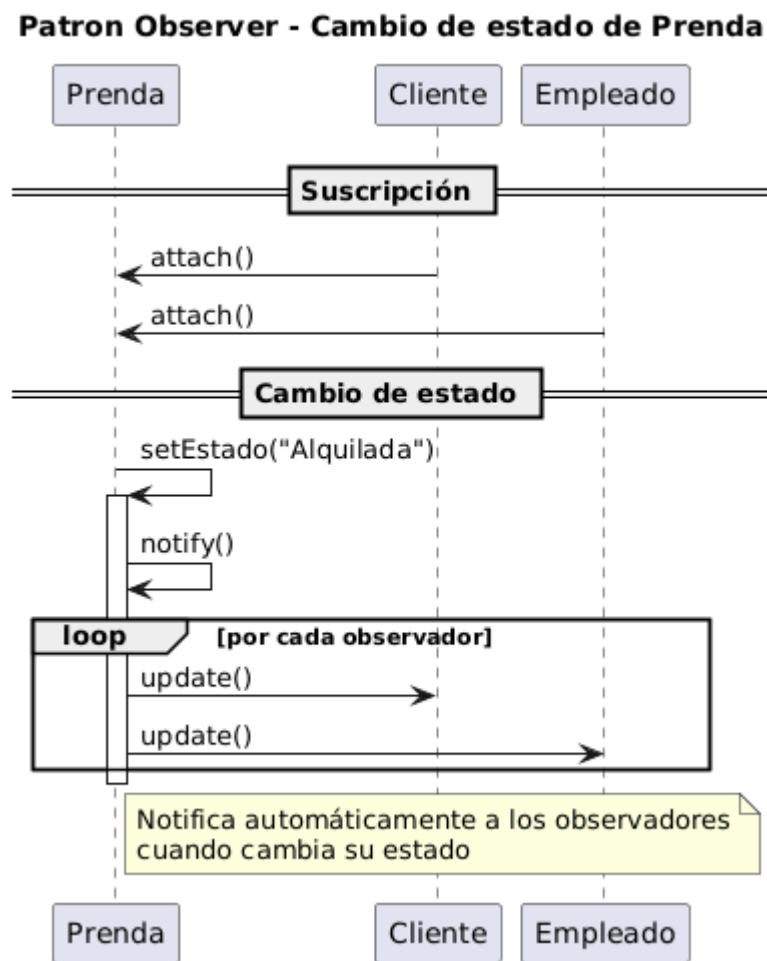


Figura 8. Diagrama de secuencia del patrón de comportamiento Strategy (Métodos de pago).

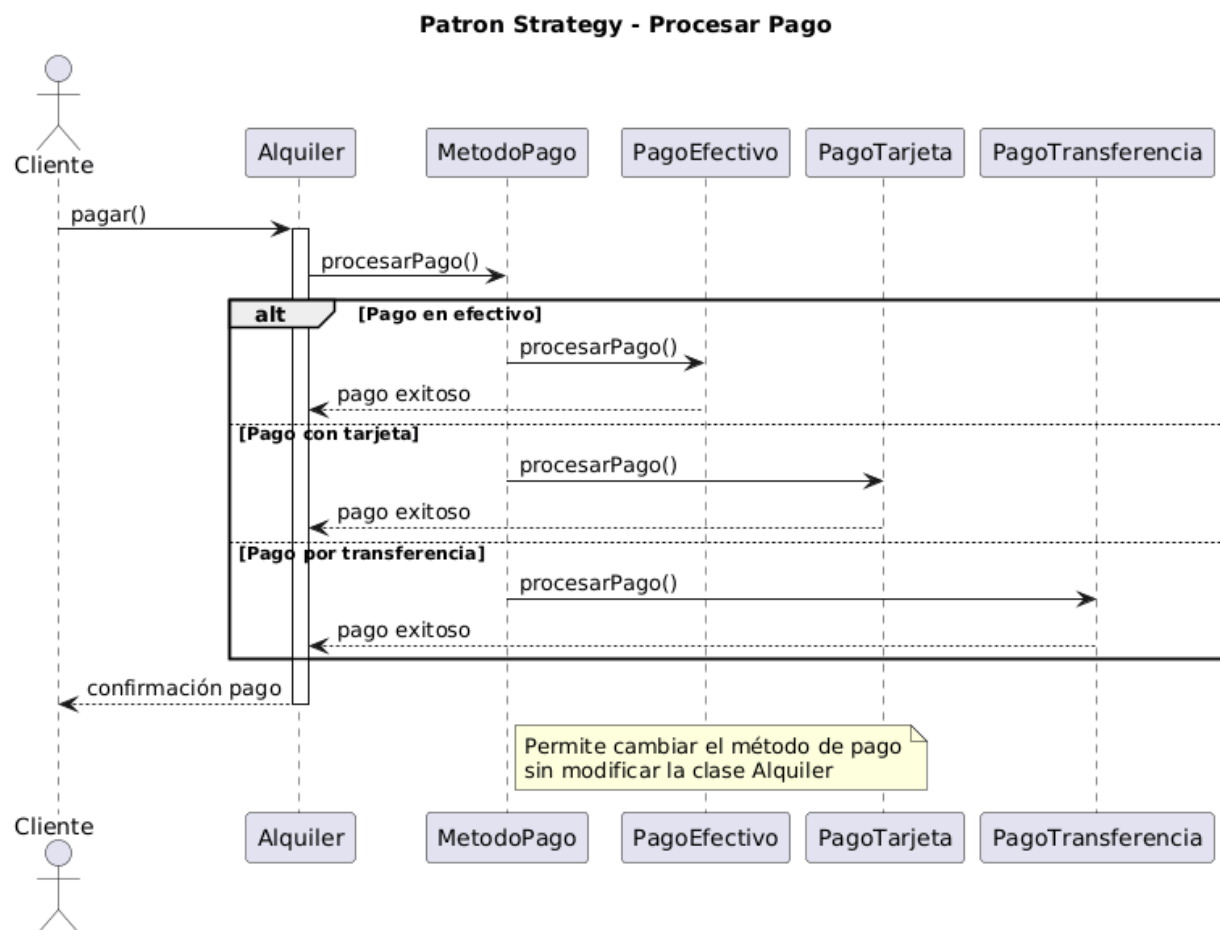
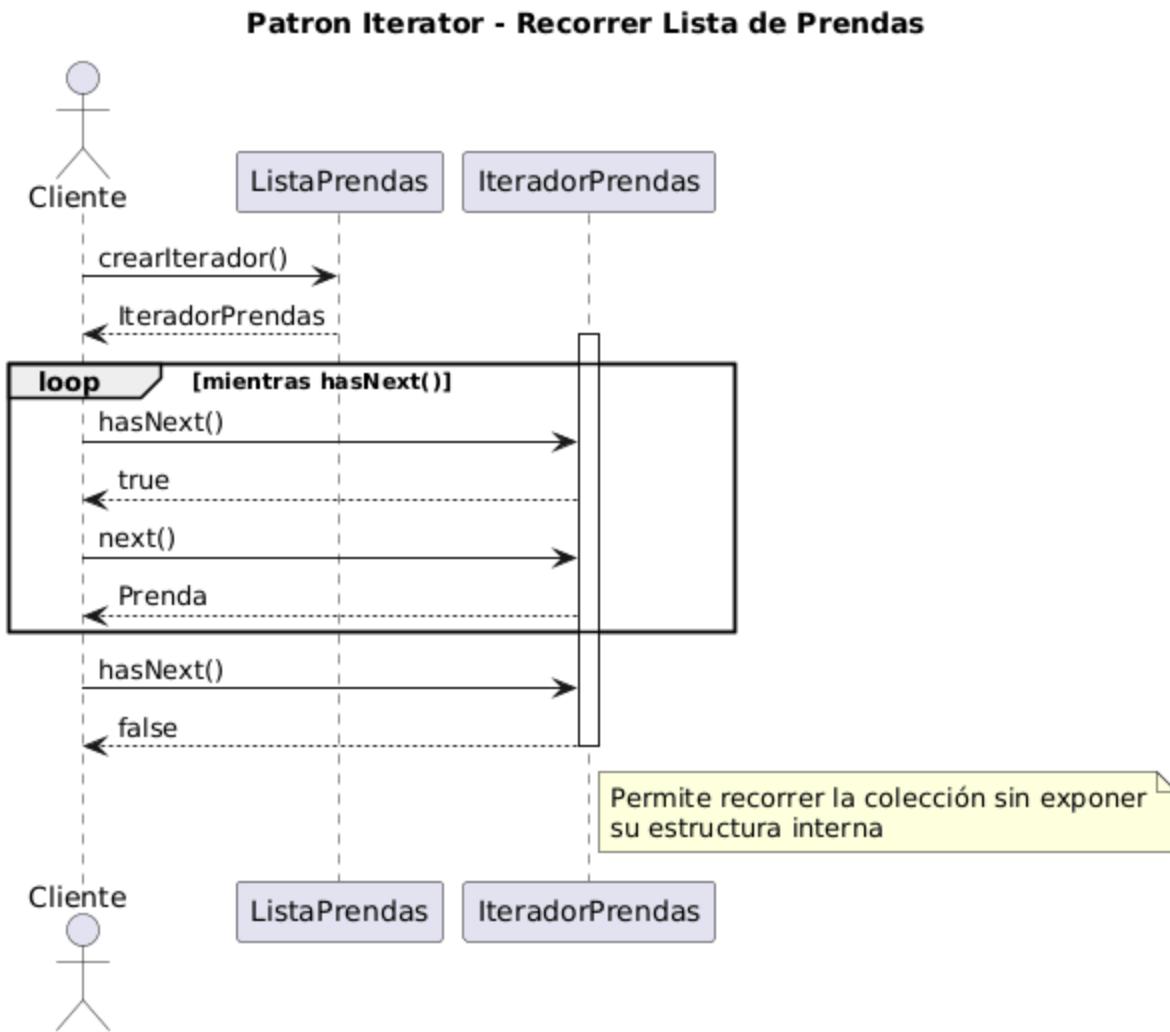


Figura 9. Diagrama de secuencia del patrón de comportamiento Iterator (recorrer prendas)



Conclusion

La aplicación de patrones de diseño en el sistema de alquiler “Los Atuendos” permitió transformar una solución inicial básica en una arquitectura más robusta, organizada y alineada con buenas prácticas de desarrollo de software. La incorporación de patrones creacionales y estructurales facilitó la correcta construcción y organización de los componentes del sistema,

mientras que los patrones de comportamiento permitieron mejorar la interacción entre los objetos y la gestión de la lógica de negocio.

Uno de los principales logros del rediseño fue la reducción del acoplamiento entre clases, lo que se traduce en una mayor facilidad para realizar cambios, mantener el sistema y escalar sus funcionalidades. Asimismo, la implementación de estrategias de pago, manejo de eventos, encapsulamiento de acciones y recorridos de colecciones evidenció una mejora significativa en la flexibilidad y reutilización del código.

Adicionalmente, la actualización de los diagramas UML permitió representar de manera más clara y precisa tanto la estructura como el comportamiento del sistema, facilitando su comprensión y validación.

En conclusión, este ejercicio demuestra que la correcta aplicación de patrones de diseño no solo mejora la calidad técnica del software, sino que también contribuye a construir soluciones más sostenibles, adaptables y preparadas para enfrentar nuevos requerimientos en el futuro.

Referencias bibliográficas

Besembel, I. (2009). TDSO: una técnica para el diseño de software orientado por objetos.

Revista Ciencia e Ingeniería. Vol. 30, No. 3, pp. 193-200. [https://elibro-](https://elibro-net.ucompensar.basesdedatosezproxy.com)

[net.ucompensar.basesdedatosezproxy.com](https://elibro-net.ucompensar.basesdedatosezproxy.com)

Debrauwer, L. (2018). Patrones de diseño en Java los 23 modelos de diseño: descripciones y

soluciones ilustradas en UML 2 et Java. Ediciones ENI. <https://www.google.com.co/books>

Díaz, J.; Harari, I. y Amadeo, A. P. (2013). Guía de recomendaciones para diseño de software

centrado en el usuario. Editorial de la Universidad Nacional de La Plata. [https://elibro-](https://elibro-net.ucompensar.basesdedatosezproxy.com)

[net.ucompensar.basesdedatosezproxy.com](https://elibro-net.ucompensar.basesdedatosezproxy.com)

Fernández, O. et al. (2011). NASPOO: una notación algorítmica estándar para Programación

Orientada a Objetos. <https://www-redalyc-org.ucompensar.basesdedatosezproxy.com>